Підготовка до заняття 3

Інтеграція LLM: API та self-hosted

Як читати цей матеріал

Це навчальний текст, а не конспект: він пояснює тему з нуля і спирається лише на заняття 1–2 (що таке токен, контекстне вікно, як модель генерує текст токен за токеном, prefill і decode). На занятті 2 ми розбирали, як модель працює всередині. Тут — як вбудувати її в продукт: як підключати модель так, щоб код не падав, скільки коштує кожен запит, звідки береться затримка і коли варто запускати модель на своєму сервері.

Читайте розділи по порядку: кожен наступний використовує попередній. Наприкінці кожного розділу є блок **«Головне»** з тим, що варто запам'ятати, і посилання **«Глибше»** для тих, хто хоче розібратися детальніше. Посилання необов'язкові. Розділ 9 — питання для самоперевірки, розділ 10 — налаштування середовища для практики. Орієнтовний час читання — 60 хвилин, налаштування — ще 30.

План

1. Від моделі до продукту
три способи підключення, OpenAI-сумісний API

2. Анатомія запиту і відповіді
messages, параметри, usage, finish_reason

3. Скільки коштує запит
вхідні й вихідні токени, tokenomics діалогу, кеш

4. Звідки береться затримка
TTFT, швидкість генерації, streaming

5. Надійність
timeout, retry, fallback, що ламається в продакшні

6. Structured output
JSON Schema, валідація, repair

7. Self-hosted
Ollama, пам'ять, коли це вигідно

8. Як обрати модель
чотири питання і чесне порівняння

9. Перевірте себе
питання для самоперевірки

10. Практика заняття
середовище, скрипт перевірки, завдання

Що ви вмітимете після заняття

- підключати LLM через API зі стійкими патернами: timeout, retry, structured output;
- рахувати вартість і latency запиту до того, як побачите рахунок;
- запускати модель локально через Ollama і чесно порівнювати її з API;
- аргументовано обирати модель під сценарій: якість, вартість, приватність, обмеження.

РОЗДІЛ 1

1 Від моделі до продукту

> Після розділу: ви знаєте три способи підключити модель і розумієте, чому один і той самий код може працювати з різними моделями.

1.1 Що змінюється, коли модель стає частиною продукту

У чаті з моделлю все просто: ви пишете питання, читаєте відповідь, і якщо вона дивна — перепитуєте. У продукті відповідь читає не людина, а код. Він розраховує, що відповідь прийде за розумний час, матиме очікуваний формат і не зробить рахунок за місяць більшим за виручку. Модель жодної з цих гарантій не дає: провайдер може бути перевантажений, відповідь може обірватися, а довгий діалог коштує непомітно дорожче за короткий.

Тому інтеграція LLM — це не один виклик функції, а невеликий шар інженерії навколо нього: обмеження часу, повтори, перевірка відповіді, облік токенів і вибір моделі під задачу. Про кожну з цих частин — окремий розділ.

1.2 Три способи підключити модель

Спосіб	Приклади	Що це означає для вас
Провайдер	OpenAI, Anthropic, Google	Модель працює на серверах розробника. Ви надсилаєте HTTP-запит і платите за кожен токен. Найсильніші моделі без власного заліза, але дані йдуть третій стороні, а ліміти й ціни встановлює провайдер.
Агрегатор	OpenRouter	Один ключ і один рахунок для сотень моделей різних провайдерів. Зручно порівнювати моделі: змінюється лише назва моделі в коді.
Self-hosted	Ollama, vLLM, LM Studio	Модель з відкритими вагами працює на вашому залізі: ноутбуці, сервері компанії чи орендованому GPU. Дані не виходять назовні, але залізо й підтримка — на вас.

1.3 OpenAI-сумісний API і `base_url`

Формат запиту OpenAI (`POST /v1/chat/completions` зі списком повідомлень) став фактичним стандартом. Його підтримують OpenRouter, Ollama, vLLM, LM Studio і багато інших. Тому той самий код на бібліотеці `openai` працює з усіма трьома способами — змінюються лише адреса сервера, ключ і назва моделі:

```
from openai import OpenAI

# провайдер через агрегатор
client = OpenAI(base_url="https://openrouter.ai/api/v1", api_key=OPENROUTER_KEY)
MODEL = "openai/gpt-4o-mini"

# та сама програма, але модель на вашому ноутбуці
client = OpenAI(base_url="http://localhost:11434/v1", api_key="ollama")
MODEL = "llama3.2:3b"
```

Саме на цьому побудована практика заняття: одна задача — три реалізації через заміну двох рядків. «Сумісний» не означає «однаковий»: окремі можливості (structured output, виклик інструментів,

кешування, поля `usage`) різні сервери підтримують по-різному. Тому після зміни моделі код варто перевірити на тих самих прикладах, а не вважати, що все працює.

ГОЛОВНЕ

Модель у продукті — це залежність, яку ви не контролюєте: вона буває повільною, недоступною, дорогою і повертає не той формат. Підключити її можна через провайдера, агрегатор або на своєму залізі, і завдяки **OpenAI-сумісному API** код для всіх трьох майже однаковий: змінюються `base_url`, ключ і модель.

Глибше: OpenRouter: Quickstart · OpenAI: Chat Completions API

РОЗДІЛ 2

2 Анатомія запиту і відповіді

> Після розділу: ви можете прочитати будь-який виклик моделі і знаєте, які поля відповіді код має перевірити.

2.1 Запит: повідомлення і ролі

Модель не має пам'яті між запитами. Усе, що вона має знати, ви передаєте в кожному запиті списком повідомлень. Кожне повідомлення має роль:

- **system** — правила для всієї розмови: хто модель, що робить, у якому форматі відповідає;
- **user** — те, що написав користувач, або дані, які треба обробити;
- **assistant** — попередні відповіді моделі. Їх передають назад, щоб модель «пам'ятала» діалог.

```
r = client.chat.completions.create(
    model=MODEL,
    messages=[
        {"role": "system", "content": "Classify the support ticket. Answer with one word."},
        {"role": "user", "content": "My parcel has not arrived for two weeks."},
    ],
    temperature=0,      # найімовірніший токен щоразу: стабільніший результат
    max_tokens=50,      # верхня межа довжини відповіді
)
```

`temperature` ви бачили на занятті 2: чим вона нижча, тим передбачуваніший вибір наступного токена. Для класифікації й екстракції ставлять 0, для генерації текстів — вище. `max_tokens` обмежує довжину відповіді: це захист і від рахунку, і від затримки.

2.2 Відповідь: три поля, які читає код

Поле	Що в ньому і навіщо
`choices[0].message.content`	Текст відповіді. Для коду — сирий рядок, який ще треба перевірити (розділ 6).
`choices[0].finish_reason`	Чому модель зупинилась. `stop` — закінчила сама. `length` — уперлася в `max_tokens`, і відповідь обрізана. Помилки API при цьому немає, тому код має перевіряти це поле сам.
`usage`	`prompt_tokens` і `completion_tokens` — скільки токенів ви надіслали і скільки отримали. З них рахують вартість кожного запиту (розділ 3).

Приклад: обірваний JSON без жодної помилки

Ви просите модель повернути JSON зі списком товарів і ставите `max_tokens=100`. Список виявився довгим, модель дійшла до ліміту посеред рядка: `{"items": [{"name": "Кава", "qty": 2}, {"name": "Мол`. Запит успішний, код статусу 200, але `json.loads` падає. Якщо перевіряти `finish_reason`, ви одразу знаєте причину: не «модель зламала формат», а «відповідь не влізла».

ГОЛОВНЕ

Кожен запит несе всю потрібну моделі інформацію: **system** — правила, **user** — дані, **assistant** — історію. У відповіді код читає не лише текст, а й **finish_reason** (чи не обрізана відповідь) і **usage** (скільки токенів оплачено).

РОЗДІЛ 3

3 Скільки коштує запит

> Після розділу: ви можете порахувати вартість запиту і місяця роботи сервісу до запуску і знаєте, що робить рахунок більшим.

3.1 Вхідні та вихідні токени

Провайдер рахує окремо токени, які ви надіслали (**input**: system prompt, історія, документи), і токени, які згенерувала модель (**output**). Ціна вказана за мільйон токенів. Вихідні зазвичай у 3–5 разів дорожчі: вхідні модель обробляє паралельно за один прохід (prefill), а вихідні генерує по одному, і кожен потребує окремого проходу (decode). Обидва терміни — з заняття 2.

```
cost = prompt_tokens / 1e6 * price_in + completion_tokens / 1e6 * price_out

# gpt-4o-mini: $0.15 за 1 млн вхідних, $0.60 за 1 млн вихідних
# запит: 800 вхідних, 200 вихідних
800 / 1e6 * 0.15 + 200 / 1e6 * 0.60 = $0.00012 + $0.00012 = $0.00024
```

Чверть цента за десять тисяч запитів здається дрібницею. Але сервіси обробляють мільйони запитів, і різниця між моделями в 10–30 разів за ціною перетворюється на різницю між «копійки» і «найбільша стаття витрат». Актуальні ціни — на openrouter.ai/models; вони змінюються, тому перед розрахунком перевіряйте.

3.2 Tokenomics діалогу: чому кожне повідомлення дорожче за попереднє

Модель нічого не пам'ятає між запитами, тому чат-бот у кожному новому запиті надсилає всю історію. Приклад: system prompt 500 токенів, кожне повідомлення користувача 100, кожна відповідь моделі 200.

Хід діалогу	1	2	3	…	10	Разом за 10 ходів
Вхідні токени запиту	600	900	1 200	…	3 300	**19 500**

Нового тексту за 10 ходів — лише 1 500 токенів (system prompt і 10 повідомлень), а оплачуємо 19 500 вхідних, тобто в 13 разів більше. Кожен хід додає до запиту ще 300 токенів, тому сума за діалог росте **квадратично** від його довжини: удвічі довший діалог коштує приблизно вчетверо дорожче.

Порахуйте на прикладі

10 000 таких діалогів на день на gpt-4o-mini: вхідні 195 млн токенів × $0.15 = $29, вихідні 20 млн × $0.60 = $12. Разом ≈ **$41 на день, ~$1 240 на місяць**. Дві третини суми — повторна передача історії.

3.3 Як зменшити рахунок

Прийом	Як працює
Обрізати історію	Передавати лише останні N ходів або стислий підсумок старих. Ризик — модель «забуде» щось важливе з початку розмови.
Коротші відповіді	Інструкція «відповідай коротко» і `max_tokens`. Вихідні токени найдорожчі, тож це найсильніший важіль.
Prompt caching	Якщо початок запиту повторюється (system prompt, інструкції, довгий документ), провайдер не обробляє його заново і бере за ці токени знижку. Кеш працює лише для однакового префікса, тому **незмінне ставимо на початок, а змінне — в кінець**. У OpenAI кеш вмикається для промптів від 1 024 токенів.
Batch API	Для фонових задач (розмітка, звіти) запити відправляють пакетом із результатом протягом доби. Близько −50% до ціни.
Дешевша модель	Простим запитам — мала модель, складним — сильна (розділ 8).

ГОЛОВНЕ

Вартість = вхідні токени × ціна + вихідні × ціна, і **вихідні в 3–5 разів дорожчі**. Діалог дорожчає з кожним ходом, бо кожен запит несе всю історію: ціна росте квадратично. Рахунок знижують обрізання історії, коротші відповіді, кеш однакового префікса і Batch API.

Глибше: OpenAI: Prompt caching · OpenAI: Batch API · OpenRouter: ціни моделей

РОЗДІЛ 4

4 Звідки береться затримка

> Після розділу: ви можете оцінити час відповіді й знаєте, що з нього реально скоротити.

4.1 З чого складається час відповіді

Мережа і черга — шлях до провайдера, очікування вільного сервера → **Prefill** — модель читає весь промпт за один прохід → **Decode** — генерація вихідних токенів по одному

Перші дві частини разом називають **TTFT** (time to first token) — час до першого токена. Далі модель генерує з певною швидкістю, яку міряють у токенах за секунду. Звідси проста формула:

```
час відповіді ≈ TTFT + вихідні токени ÷ швидкість генерації

# TTFT 0.4 с, відповідь 300 токенів, 60 токенів/с
0.4 + 300 / 60 = 0.4 + 5.0 = 5.4 с
```

У цьому прикладі генерація займає 93% часу. Скоротити відповідь удвічі — мінус 2.5 секунди. Скоротити промпт удвічі — мінус частки секунди, бо prefill швидкий. Тому **довжина виходу майже завжди важливіша за довжину входу**. Виняток — дуже довгі входи (сотні сторінок): там prefill теж помітний.

4.2 Streaming: та сама швидкість, інше відчуття

З параметром `stream=True` відповідь приходить частинами в міру генерації. Загальний час не змінюється, але користувач бачить перші слова через TTFT (~0.5 с), а не через 5 секунд. Для чату це обов'язково. Для фонової обробки, де результат читає код, — не потрібно: код однаково чекатиме кінця, а потік ускладнює обробку помилок і JSON.

4.3 Як чесно міряти

- **Не по одному запиту.** Latency скаче від запиту до запиту. Беріть 5–20 запусків і дивіться медіану, а для SLA — 95-й перцентиль (p95).
- **Відкидайте холодний старт.** Перший запит до локальної моделі повільний, бо вона вантажиться в пам'ять: у нас `llama3.2:3b` на MacBook відповіла за 5.7 с уперше і за 0.1 с наступного разу.
- **Порівнюйте на однакових даних.** Відповідь на інший текст має іншу довжину, а отже інший час.

ГОЛОВНЕ

Час відповіді ≈ **TTFT + вихідні токени ÷ швидкість**, і найбільша частина — генерація. Головний важіль — коротша відповідь; для чатів — streaming, що не прискорює, але показує перші слова одразу. Міряйте медіану кількох запусків і без холодного старту.

Глибше: OpenAI: Latency optimization — сім принципів · OpenAI: Streaming

---

РОЗДІЛ 5

5 Надійність: патерни інтеграції

> Після розділу: ви знаєте мінімальний набір захистів для будь-якого виклику моделі і як відрізнити помилку, яку варто повторити, від тієї, яку треба виправляти.

5.1 Timeout

Без обмеження часу запит до перевантаженого провайдера може «висіти» хвилинами: ваш сервіс тримає з'єднання, користувач чекає, а нові запити накопичуються. Timeout — максимальний час очікування, після якого запит вважається невдалим. Значення підбирають за реальною latency моделі з запасом: якщо медіана 5 с, а p95 — 12 с, розумний timeout — 30 с.

```
client = OpenAI(base_url=BASE_URL, api_key=API_KEY, timeout=30, max_retries=2)
```

5.2 Retry з exponential backoff

Провайдер обмежує кількість запитів і токенів за хвилину (**rate limit**). Коли ліміт перевищено, він повертає помилку `429`. Тимчасові збої — це також `500`, `502`, `503` і обрив з'єднання. Правильна реакція на них — повторити запит, щоразу чекаючи довше: 1 с, 2 с, 4 с… (**exponential backoff**) плюс

випадкова добавка (**jitter**). Без jitter тисячі клієнтів, що отримали помилку одночасно, одночасно і повторять запит — і знову перевантажать сервер.

Помилка	Що означає	Повторювати?
`429`	перевищено ліміт запитів або токенів	так, з backoff
`500 / 502 / 503`, timeout	збій на стороні провайдера	так, 2–3 спроби
`400`	помилка у вашому запиті: неправильний параметр, задовгий контекст	ні — виправляти код або дані
`401 / 403`	ключ невірний або без доступу до моделі	ні

SDK OpenAI вже робить 2 повтори з backoff за замовчуванням (`max_retries`). Повторювати `400` марно: той самий запит отримає ту саму помилку, а ви заплатите часом.

5.3 Fallback

Якщо провайдер недоступний довше, ніж тривають повтори, запит переводять на запасну модель або іншого провайдера. Агрегатори на кшталт OpenRouter вміють це робити самі. Важливо: запасна модель має бути перевірена на ваших прикладах заздалегідь, інакше fallback замінює «немає відповіді» на «неправильну відповідь».

5.4 Що ламається в продакшні, хоча помилки немає

Ситуація	Що робити
Обрізана відповідь	`finish_reason == "length"`: JSON зламаний, хоча API відповів успішно. Перевіряйте завжди; повторюйте з більшим `max_tokens` або просіть коротший вихід.
Не влазить контекст	`400` через довжину повтором не лікується. Обрізайте історію або діліть документ на частини.
Невалідна відповідь	Повтор з текстом помилки для моделі — repair loop (розділ 6.3). Кожна спроба коштує, тому ліміт спроб — 1–2.
Модель тихо змінилась	Провайдери оновлюють модель за аліасом (`gpt-4o`) і виводять старі з обігу. Фіксуйте snapshot-назву з датою (`gpt-4o-2024-08-06`), а після зміни — проганяйте свої тести.

ГОЛОВНЕ

Мінімум для будь-якого виклику моделі: **timeout**, **retry з backoff і jitter** лише для 429, 5xx і обривів, перевірка **finish_reason**. Для продакшну — ще fallback і зафіксована версія моделі. Правило: **модель пропонує — код приймає**. Усе, що можна перевірити кодом, перевіряйте кодом.

Глибше: OpenAI Cookbook: How to handle rate limits · AWS Builders' Library: Timeouts, retries and backoff with jitter

РОЗДІЛ 6

6 Structured output

> Після розділу: ви знаєте, як отримати від моделі відповідь у форматі, який код може надійно прочитати, і що робити, коли формат правильний, а дані — ні.

6.1 Навіщо JSON Schema

Коли відповідь читає код, вільний текст не годиться: «Категорія: доставка, пріоритет високий» доведеться розбирати регулярками, і вони зламаються на першому «Категорія — Доставка». Тому модель просять повернути JSON за схемою. Найнадійніший спосіб — передати схему в API через `response_format` з `"strict": true`: тоді модель під час генерації не може видати токен, що ламає схему. Схему зручно описати класом Pydantic:

```
from typing import Literal
from pydantic import BaseModel

class Ticket(BaseModel):
    category: Literal["delivery", "payment", "refund", "technical", "other"]
    priority: Literal["low", "medium", "high"]
    order_id: str | None          # None, якщо в зверненні немає номера

r = client.chat.completions.create(
    model=MODEL, messages=messages, temperature=0,
    response_format={"type": "json_schema", "json_schema": {
        "name": "ticket", "strict": True, "schema": Ticket.model_json_schema()}},
)
ticket = Ticket.model_validate_json(r.choices[0].message.content)
```

`Literal` обмежує значення закритим списком: модель не придумає категорію «логістика». `str | None` дозволяє відповісти «немає»: без цього модель змушена щось вигадати.

6.2 Правильна структура ≠ правильні дані

Схема гарантує форму: всі поля на місці, типи правильні, значення зі списку. Але вона не гарантує, що `order_id` справді є в тексті звернення, а категорія обрана правильно. Тому відповідь завжди проходить **валідацію в коді**: Pydantic перевіряє типи, а ваші правила — факти (номер замовлення має бути в тексті, сума не від'ємна).

Що ламається

Не всі моделі й сервери підтримують strict-схему. Малі й локальні моделі частіше повертають JSON у markdown-блоці, з зайвим текстом чи пропущеним полем. На практиці порахуємо, як часто.

Відмова (refusal)

Модель може відмовитись відповідати через політику безпеки. У SDK OpenAI відмова приходить в окремому полі `message.refusal`, а не в JSON. Код перевіряє його першим.

6.3 Repair: план на помилку валідації

Якщо відповідь не пройшла валідацію, є три варіанти: повторити запит, додавши текст помилки («field priority: value 'urgent' is not one of low, medium, high»), перейти на сильнішу модель або

позначити запис для перевірки людиною. Повтор з текстом помилки (**repair**) часто спрацьовує з першого разу, але кожна спроба коштує, тому кількість спроб обмежують.

ГОЛОВНЕ

Відповідь для коду — це JSON за схемою: **response_format strict** гарантує структуру, **Literal** і **None** зменшують простір для вигадок. Структура не гарантує правильних даних, тому відповідь завжди проходить **валідацію в коді**, а на помилку є план: repair, інша модель або людина.

Глибше: OpenAI: Structured outputs · Pydantic: Models · Ollama: Structured outputs

---

РОЗДІЛ 7

7 Self-hosted: модель на своєму залізі

> Після розділу: ви можете запустити локальну модель, оцінити, яке залізо їй потрібне, і порахувати, коли self-hosted справді вигідніший за API.

7.1 Відкриті ваги і Ollama

Моделі з відкритими вагами (Llama, Qwen, Gemma, Mistral, DeepSeek) можна завантажити і запустити самостійно. **Ollama** — найпростіший спосіб: одна команда завантажує модель, і вона одразу доступна через OpenAI-сумісний API на `localhost:11434`. Для продакшн-навантаження з багатьма паралельними запитами частіше беруть **vLLM** — сервер, оптимізований під пропускну здатність GPU.

```
ollama pull llama3.2:3b                # ~2 ГБ
ollama run llama3.2:3b "Привіт"        # перевірка в терміналі
```

7.2 Скільки потрібно пам'яті

Модель під час роботи має повністю лежати в пам'яті (RAM або відеопам'ять GPU). Груба оцінка: **кількість параметрів × байтів на параметр**. В оригіналі параметр займає 2 байти (16 біт), тому 7B-модель важить ~14 ГБ. Ollama за замовчуванням завантажує **квантовані** моделі: ваги стиснуті до ~4 біт, тобто ~0.5–0.6 байта на параметр. Звідси `llama3.2:3b` — 2 ГБ, `qwen2.5:7b` — 4.7 ГБ. Квантування трохи знижує якість, але дає змогу запустити модель на ноутбуці.

Модель	Розмір у Ollama (4 біт)	Комфортно працює на
3B	~2 ГБ	ноутбук з 8 ГБ пам'яті
7–8B	~5 ГБ	ноутбук з 16 ГБ, Apple Silicon
70B	~40 ГБ	сервер з GPU на 48+ ГБ

7.3 Коли self-hosted виправданий, а коли це ілюзія економії

Виправдано

- Дані не можна передавати третій стороні: медицина, банки, договори клієнтів, air-gapped середовища.
- Великий рівний потік простих однотипних запитів, з якими справляється мала модель.
- Власна дотренована (fine-tuned) модель або повний контроль над версією.

Ілюзія економії

- GPU-сервер коштує грошей щогодини, навіть коли запитів немає.
- До ціни заліза додаються налаштування, моніторинг, оновлення і час інженера.
- Мала модель на складній задачі дає більше помилок, і їх виправлення теж коштує.

Порахуйте на прикладі

Орендований GPU рівня L4 / A10 — умовно $0.7–1 за годину, тобто **$500–720 на місяць** цілодобової роботи. За $720 через API gpt-4o-mini можна обробити приблизно 4.8 млрд вхідних токенів. Якщо ваш сервіс споживає 50 млн токенів на місяць, API обійдеться в ~$10, а сервер — у ті самі $720.

Висновок: self-hosted стає дешевшим лише при великому і рівномірному навантаженні. Для більшості MVP API вигідніший, а локальну модель обирають з міркувань приватності, а не ціни.

Приватність без свого заліза. Між «усе до OpenAI» і «свій сервер» є середній варіант: моделі через хмару, з якою компанія вже має договір, — Azure OpenAI, AWS Bedrock, Google Vertex AI. Дані не йдуть на навчання моделей, а регіон зберігання обираєте ви.

ГОЛОВНЕ

Self-hosted — це модель з відкритими вагами на вашому залізі; **Ollama** запускає її однією командою з тим самим API. Пам'ять ≈ параметри × байти на параметр, квантування стискає модель у ~4 рази. Self-hosted виправданий для **приватності й контролю**; дешевшим він стає лише при великому рівному навантаженні.

Глибше: Ollama: OpenAI compatibility · vLLM · Hugging Face: Quantization

РОЗДІЛ 8

8 Як обрати модель

> Після розділу: ви можете обґрунтувати вибір моделі цифрами на своїх даних, а не враженням чи бенчмарком.

8.1 Чотири питання

Критерій	Що питаємо	Чим міряємо
Якість	Чи справляється модель на ваших реальних прикладах? Бенчмарки — лише орієнтир: вони міряють загальні задачі, а не вашу.	20–50 розмічених прикладів
Вартість	Ціна запиту × очікувана кількість запитів на місяць. Рахуйте до запуску.	$ на місяць
Приватність	Які дані йдуть у запит і чи можна віддавати їх цьому провайдеру?	NDA, персональні дані
Обмеження	Latency, rate limits, розмір контексту, підтримка structured output.	медіана і p95, ліміти

8.2 Чесне порівняння

Порівняння має сенс лише за однакових умов: ті самі приклади, той самий промпт, `temperature=0`, кілька запусків, без холодного старту. Невалідні відповіді не викидають зі звіту, а рахують окремо: модель, що в 3 з 50 випадків ламає JSON, у продакшні в стільки ж випадках не дасть результату. Результат зводять у таблицю трейд-офів:

Модель	Час (медіана)	$ / 1000 запитів	Валідний JSON	Правильна категорія
Провайдерська (еталон)	…	…	…/50	…/50
Дешевша через API	…	…	…/50	…/50
Локальна	…	$0 + залізо	…/50	…/50

Саме таку таблицю ми заповнимо на практиці, а ви зробите її для свого курсового проєкту на кроці 2 кейсу 1.

8.3 Одна модель на все — рідкість

Частіше рішення виглядає так: дешева модель для простих і масових запитів, сильна — для складних або коли дешева не впоралась (**routing**, **cascade**). Наприклад, класифікацію звернень робить мала модель, а відповідь клієнту на складну скаргу пише сильна.

ГОЛОВНЕ

Модель обирають за чотирма критеріями: **якість на ваших прикладах, вартість на місяць, приватність даних, обмеження**. Порівнюють на однакових даних і умовах, невалідні відповіді рахують окремо. Часто найкраще рішення — комбінація дешевої і сильної моделі.

Глибше: Artificial Analysis: якість, ціна і швидкість моделей · LMArena: рейтинг моделей

РОЗДІЛ 9

9 Перевірте себе

> Спробуйте відповісти без підглядання. Якщо питання викликає складність, поверніться до розділу в дужках.

1. Що потрібно змінити в коді, щоб замість gpt-4o-mini через OpenRouter використати локальну модель в Ollama? Що варто перевірити після зміни? *(1.3)*
2. API повернув статус 200, але `json.loads` падає. Яке поле відповіді ви перевірите першим і що воно може показати? *(2.2)*
3. Чому 10-те повідомлення в чаті коштує дорожче за перше, хоча воно такої ж довжини? *(3.2)*
4. Чому вихідні токени дорожчі за вхідні? *(3.1)*
5. Як треба впорядкувати частини промпту, щоб спрацював prompt caching? *(3.3)*
6. Що сильніше зменшить час відповіді: скоротити промпт удвічі чи відповідь удвічі? Чому? *(4.1)*
7. Локальна модель відповіла за 6 с, а API — за 1.5 с. Назвіть дві можливі причини, не пов'язані з «локальна повільніша». *(4.3)*
8. На які помилки API варто робити retry, а на які — ні? Навіщо до backoff додають jitter? *(5.2)*
9. Схема з `strict: true` гарантує, що `order_id` правильний? Що робити, щоб це перевірити? *(6.2)*
10. Скільки пам'яті приблизно потрібно 7B-моделі без квантування і з 4-бітним квантуванням? *(7.2)*
11. Коли self-hosted справді дешевший за API, а коли лише здається дешевшим? *(7.3)*
12. Які дані з вашого курсового проєкту не можна відправляти сторонньому провайдеру? Які варіанти лишаються? *(7.3, 8.1)*

РОЗДІЛ 10

10 Практика заняття

> На практиці ви запускатимете один скрипт з API провайдера і з локальною моделлю. Налаштування займає ~30 хвилин; зробіть його заздалегідь.

Python, бібліотеки і ключ

Потрібен Python 3.10 або новіший (`python3 --version`). Ключ OpenRouter з заняття 2 підійде; на балансі достатньо кількох доларів, практика коштує центи.

```
pip install openai python-dotenv pydantic

# .env — ключ тут, а не в коді
OPENROUTER_API_KEY=sk-or-...

# .gitignore — щоб ключ не потрапив у репозиторій
.env
```

Ollama і модель

Встановіть Ollama з ollama.com/download (macOS, Windows, Linux) і завантажте модель заздалегідь: це 2 ГБ, на занятті мережа буде перевантажена. Потрібно від 8 ГБ оперативної пам'яті.

```
ollama pull llama3.2:3b
ollama run llama3.2:3b "Привіт"
```

Немає можливості встановити локально? Пройдіть практику в Google Colab, як на занятті 2: ключ у Secrets, Ollama запускається прямо в ноутбуці. Але локальне середовище краще: далі в курсі ми працюємо зі скриптами і репозиторієм.

Скрипт перевірки `check.py`

Скрипт надсилає однакове питання в OpenRouter і в локальну Ollama тим самим кодом. Різниця лише в `base_url`, ключі й назві моделі — так влаштована і практика.

```
import os, time
from dotenv import load_dotenv
from openai import OpenAI

load_dotenv()
QUESTION = "What is 2 + 2? Answer with a single number."
TARGETS = [
    ("OpenRouter", "https://openrouter.ai/api/v1",
     os.getenv("OPENROUTER_API_KEY"), "openai/gpt-4o-mini"),
    ("Ollama", "http://localhost:11434/v1", "ollama", "llama3.2:3b"),
]

for name, base_url, key, model in TARGETS:
    if not key:
        print(f"{name}: OPENROUTER_API_KEY is missing in .env")
        continue
    client = OpenAI(base_url=base_url, api_key=key, timeout=60, max_retries=2)
    start = time.perf_counter()
    try:
        r = client.chat.completions.create(
            model=model, temperature=0,
            messages=[{"role": "user", "content": QUESTION}])
    except Exception as e:
        print(f"{name}: error {type(e).__name__}: {e}")
        continue
    sec = time.perf_counter() - start
    u = r.usage
    print(f"{name} | {model} | {sec:.1f}s | "
          f"tokens: in {u.prompt_tokens}, out {u.completion_tokens} | "
          f"{r.choices[0].message.content.strip()}")
```

Що побачите

```
OpenRouter | openai/gpt-4o-mini | 0.?s | tokens: in ?, out ? | 4
Ollama | llama3.2:3b | 0.1s | tokens: in 39, out 2 | 4
```

Цифри у вас будуть інші. Перший запит до Ollama повільний, бо модель вантажиться в пам'ять: запустіть скрипт удруге і порівняйте (розділ 4.3).

Що робимо на занятті

Етап	Що відбувається
Практика	Скрипт з репозиторію курсу (`lesson-03-llm-integration`) класифікує 5 звернень клієнтів і повертає JSON. Ви допишете structured output, валідацію, підрахунок вартості й timeout (TODO 3–6), поміняєте `base_url` і модель і напишете в чат: модель, час, вартість, скільки з 5 відповідей — валідний JSON. Разом зведемо таблицю трейд-офів (розділ 8.2).
Кейс 1, крок 2	Друга модель і заміри якості, вартості й latency на задачі вашого проєкту: той самий скрипт, ваша схема і ваші тексти.

Питання по теорії пишіть до заняття в тред Slack «Питання по теорії до заняття 3». Якщо питання вже є, поставте йому реакцію: так видно, що цікаво більшості.